

新哲文院算法社 2026-2027 学年 C++ 教材

新哲文院算法社 2026-2027 学年 C++ 教材

适用对象：零基础学员，从安装到竞赛进阶

教材定位：课堂讲义 + 自学参考

配套文件：[C++教材_答案与解析.md](#)

教材结构说明

本教材分为两大模块：

模块	内容	目标
基础掌握	环境搭建 → 语法基础 → 数据结构 → 函数 → 文件与异常	掌握 C++ 编程基本功
竞赛进阶	排序算法 → 搜索算法 → 递归与递推	具备算法竞赛入门能力

第一部分：基础掌握

第一单元：语言基础与环境搭建

1.1 编程语言概述

什么是编程语言

编程语言是人与计算机沟通的"桥梁"。计算机只能理解二进制（0 和 1），而人类需要一种更直观的方式来下达指令——这就是编程语言的作用。

编译型 vs 解释型

类型	原理	代表语言	优点	缺点
编译型	源代码一次性翻译为机器码，生成可执行文件	C、C++	运行速度快	编译耗时，跨平台需重新编译
解释型	逐行读取源代码，边翻译边执行	Python、JavaScript	跨平台性好，开发灵活	运行速度相对较慢

C++ 是编译型语言：你写的 `.cpp` 源代码必须先经过编译器（如 `g++`）编译成 `.exe`（Windows）或可执行文件（Linux/Mac），然后才能运行。这也是 C++ 运行效率极高的原因。

为什么选择 C++ 学算法竞赛

- 运行速度极快：接近底层硬件，适合大规模数据
- STL 标准库强大：`vector`、`map`、`set`、`queue` 等开箱即用
- 竞赛主流语言：NOIP、CSP、ICPC、Codeforces 均首选 C++
- 内存可控：指针和引用让你可以精细管理内存

1.2 开发环境配置

安装编译器（MinGW-w64）

C++ 需要编译器才能运行。Windows 下最常用的是 **MinGW-w64**（GCC 的 Windows 移植版）。

安装步骤：

1. 下载 MinGW-w64 安装程序：<https://sourceforge.net/projects/mingw-w64/files/>
2. 运行安装程序，关键选项设置：

选项	推荐选择	说明
Version	最高版本	如 13.2.0
Architecture	x86_64（64 位系统）	32 位选 i686
Threads	win32（开发 Windows 程序）	Linux 开发选 posix
Exception	seh（64 位推荐）	32 位选 dwarf
Build revision	默认即可	—

- 3. 安装路径必须**全英文、无空格**（如 `C:\mingw64`）
- 4. 等待下载完成（可能需要较长时间）

配置环境变量：

- 5. 右击"此电脑" → 属性 → 高级系统设置 → 环境变量
- 6. 在"系统变量"中找到 `Path` → 编辑 → 新建
- 7. 添加 MinGW-w64 的 `bin` 目录路径（如 `C:\mingw64\bin`）
- 8. 确认保存

验证安装：

打开命令提示符（Win+R → 输入 `cmd`），执行：

```
Bash
1 gcc -v
```

出现版本信息即配置成功。

安装 Visual Studio Code（VSCode）

VSCode 是微软推出的轻量级代码编辑器，配合插件可以实现强大的 C++ 开发功能。

安装步骤：

- 1. 访问官网：<https://code.visualstudio.com/> 下载安装包
- 2. 运行安装程序，安装路径建议全英文、无空格
- 3. 安装时勾选"添加到 PATH"和"右键菜单集成"

配置 C++ 开发环境：

步骤	操作	说明
① 安装中文插件	Ctrl+Shift+X → 搜索 Chinese → 安装	界面汉化
② 安装 C/C++ 插件	扩展面板搜索 C/C++ → 安装 Microsoft 官方插件	代码补全、调试
③ 配置编译器路径	Ctrl+Shift+P → 搜索 C/C++: Edit Configurations (UI) → 设置编译器路径	告诉 VSCode 编译器在哪
④ 配置构建任务	Ctrl+Shift+P → 搜索 Tasks: Configure Default Build Task → 选择 g++.exe	告诉 VSCode 如何编译
⑤ 配置调试	按 F5 → 选择 C++ (GDB/LLDB) → 选择 g++.exe	配置调试环境

 参考教程：C++ VSCode 环境配置保姆级教程：https://blog.csdn.net/qq_45807140/article/details/112862592

实用快捷键：

快捷键	功能
Shift + Alt + F	格式化代码
Ctrl + /	快速注释 / 取消注释
F5	启动调试
Ctrl + Shift + P	全局命令面板

 **小技巧：**配置成功后，把 .vscode 文件夹备份一份，以后新建项目直接复制进去即可，无需重复配置。

1.3 第一个程序：Hello World

编写与运行

在 VSCode 中创建文件 `hello.cpp`：

C/C++

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout << "Hello World" << endl;
6     return 0;
7 }
```

运行方式：

- 方法一：右键编辑器 → 在终端中运行（需配置好构建任务）
- 方法二：终端手动编译运行：

Bash

```
1 g++ hello.cpp -o hello
2 ./hello
```

- 方法三：按 **F5** 启动调试运行

输出：

Plain Text

```
1 Hello World
```

代码解析

代码	作用
<code>#include <iostream></code>	引入输入输出流库（cout/cin 需要）
<code>using namespace std;</code>	使用标准命名空间，省去 <code>std::</code> 前缀
<code>int main()</code>	程序入口函数，每个 C++ 程序必须有
<code>cout << "Hello World" << endl;</code>	输出字符串并换行
<code>return 0;</code>	返回 0 表示程序正常结束

竞赛标准模板

C/C++

```
1 #include <bits/stdc++.h>    // 万能头文件：一次性引入几乎所有常用库
2 using namespace std;
3
4 int main() {
5     // 你的代码写在这里
6     ios::sync_with_stdio(false); // 关闭同步，加速 cin/cout
7     cin.tie(0);                  // 解绑 cin 和 cout
8
9     return 0;
10 }
```

 **万能头文件** `<bits/stdc++.h>` 是 GCC 编译器提供的非标准头文件，竞赛中极其常用——一行搞定所有包含，省时省力。但注意：部分在线判题系统可能不支持。

输入输出基础

`cin` 输入：

C/C++

```
1 int a;
2 cin >> a;           // 输入一个整数
3
4 int b, c;
5 cin >> b >> c;      // 连续输入多个变量
6
7 string s;
8 cin >> s;           // 输入字符串（遇空格停止）
9
10 double d;
11 cin >> d;          // 输入浮点数
```

cout 输出：

C/C++

```
1 int a = 10;
2 cout << a << endl;           // 输出：10（换行）
3 cout << "a = " << a << endl; // 输出：a = 10
4 cout << "Hello" << " " << "World" << endl; // 拼接输出
```

scanf / **printf**（竞赛常用，速度更快）：

C/C++

```
1 int a;
2 double b;
3 scanf("%d", &a);           // 输入整数（注意 & 取地址符）
4 scanf("%lf", &b);          // 输入浮点数（double 用 %lf）
5
6 printf("a = %d\n", a);      // 输出整数
7 printf("b = %.2f\n", b);    // 输出浮点数（保留2位小数）
```

常用格式符对照表：

格式符	用途	示例
%d	int 整数	scanf("%d", &a)
%lld	long long	scanf("%lld", &x)
%f	float 输入	scanf("%f", &f)
%lf	double 输入（必须）	scanf("%lf", &d)
%c	字符	scanf("%c", &ch)
%s	字符串	scanf("%s", s)
%.nf	保留 n 位小数输出	printf("%.2f", pi)

⚠ 常见错误：

忘记 & 取地址符：scanf("%d", a) ❌ → scanf("%d", &a) ✅

double 输入用 %f：scanf("%f", &d) ❌ → scanf("%lf", &d) ✅

单元练习 1.1

练习：编写一个程序，输入两个整数 a 和 b，输出它们的和、差、积、商（整除）。

输入示例：

10 3

输出示例：

和=13 差=7 积=30 商=3

第二单元：变量、数据类型与运算符

2.1 变量与常量

变量的声明与赋值

C++ 是静态类型语言，变量必须先声明类型才能使用：

C/C++

```
1 int a = 10;           // 整数
2 double pi = 3.14159; // 浮点数
3 char ch = 'A';        // 字符（单引号）
4 string name = "Alice"; // 字符串（双引号）
5 bool is_student = true; // 布尔值
6
7 // 多个变量同时声明
8 int x = 1, y = 2, z = 3;
9
10 // 交换两个变量的值
11 int temp = a;
12 a = b;
13 b = temp;
```

命名规则与命名规范

硬性规则（必须遵守）：

规则	正确示例	错误示例
只能包含字母、数字、下划线	my_var	my-var
不能以数字开头	var1	1var
不能使用关键字	my_class	class
区分大小写	Age 和 age 不同	—

命名规范（建议遵守）：

- 蛇形命名法（snake_case）：变量名用小写字母 + 下划线，如 student_name
- 见名知意：a = 10 ❌ → age = 10 ✅
- 常量名全大写：MAX_SIZE = 100

常量的定义

C/C++

```
1 // 方式一: const (推荐)
2 const int MAXN = 100005;
3 const double PI = 3.1415926535;
4 const int INF = 0x3f3f3f3f; // 竞赛常用"无穷大"值
5
6 // 方式二: #define 宏定义
7 #define MAXN 100005
8 #define INF 0x3f3f3f3f
```

💡 `0x3f3f3f3f` 是竞赛中常用的"无穷大"值：约 $1e9$ ，两个数相加不会溢出 `int`，且任何合法值都小于它。

2.2 基本数据类型

整数类型

类型	字节数	范围
<code>int</code>	4 字节	-2,147,483,648 ~ 2,147,483,647
<code>long long</code>	8 字节	-9e18 ~ 9e18
<code>short</code>	2 字节	-32,768 ~ 32,767
<code>char</code>	1 字节	-128 ~ 127（也用于字符）

C/C++

```
1 int a = 100;
2 long long b = 1234567890123LL; // LL 后缀表示 long long 字面量
3 short c = 30000;
4 char d = 'A';
5
6 // 八进制和十六进制
7 int hex = 0x64; // 十六进制: 100
8 int oct = 0144; // 八进制: 100
```

 **竞赛提醒：**当数据范围可能超过 2e9 时，务必使用 `long long`，否则会溢出！

浮点类型

类型	字节数	精度	适用场景
<code>float</code>	4 字节	6~7 位有效数字	精度要求不高
<code>double</code>	8 字节	15~16 位有效数字	默认推荐

C/C++

```
1 float f = 3.14f; // f 后缀表示 float
2 double d = 3.1415926535;
3
4 // 输出保留小数
5 printf("%.2f\n", d); // 保留2位: 3.14
6 cout << fixed << setprecision(2) << d << endl; // 同样保留2位
```

布尔类型

C/C++

```
1 bool flag = true;    // 真
2 bool flag2 = false;  // 假
3
4 // 非零值自动转为 true, 零值转为 false
5 if (5) cout << "true" << endl;    // 输出 true
6 if (0) cout << "no" << endl;      // 不执行
```

隐式类型转换

C/C++

```
1 int a = 5;
2 double b = 2.0;
3
4 double result = a / b;    // a 被隐式转换为 double → 2.5
5 int result2 = a / 2;      // 整数除法, 结果 2 (截断小数)
```

显式类型转换（强制转换）

C/C++

```
1 int a = 5, b = 2;
2
3 // C 风格 (竞赛常用)
4 double r1 = (double)a / b;    // 2.5
5
6 // C++ 风格
7 double r2 = double(a) / b;    // 2.5
8
9 // ⚠ 常见错误: 整数除法
10 cout << 5 / 2;    // 输出 2 (整除)
11 cout << 5.0 / 2;  // 输出 2.5
```

2.3 运算符

算术运算符

C/C++

```
1 int a = 10, b = 3;
2
3 cout << a + b;    // 加法：13
4 cout << a - b;    // 减法：7
5 cout << a * b;    // 乘法：30
6 cout << a / b;    // 整除：3（丢弃小数！）
7 cout << a % b;    // 取余：1
8 cout << a++;      // 后置自增：先返回10，再变成11
9 cout << ++a;      // 前置自增：先变成12，再返回12
```

运算符	含义	示例
+	加法	a + b
-	减法	a - b
*	乘法	a * b
/	除法（整数除法截断）	a / b
%	取余（模运算）	a % b
++	自增	a++ 或 ++a
--	自减	a-- 或 --a

比较运算符

运算符	含义	示例
<code>==</code>	等于	<code>a == b</code>
<code>!=</code>	不等于	<code>a != b</code>
<code><</code>	小于	<code>a < b</code>
<code>></code>	大于	<code>a > b</code>
<code><=</code>	小于等于	<code>a <= b</code>
<code>>=</code>	大于等于	<code>a >= b</code>

C/C++

```
1 int a = 5, b = 3;
2 cout << (a > b);    // 输出 1 (true)
3 cout << (a == b);   // 输出 0 (false)
```

逻辑运算符

运算符	含义	示例
<code>&&</code>	逻辑与（并且）	<code>a > 0 && b > 0</code>
<code> </code>	逻辑或（或者）	<code>a > 0 b > 0</code>
<code>!</code>	逻辑非（取反）	<code>!(a > 0)</code>

C/C++

```
1 int a = 5, b = -3;
2
3 if (a > 0 && b > 0) {
4     cout << "都大于0" << endl;
5 }
6
7 if (a > 0 || b > 0) {
8     cout << "至少有一个大于0" << endl; // 会执行
9 }
10
11 if (!(a < 0)) {
12     cout << "a不小于0" << endl;
13 }
```

💡 短路求值：`&&` 左侧为 false 时右侧不执行；`||` 左侧为 true 时右侧不执行。

赋值运算符与复合赋值

C/C++

```
1 int a = 10;
2 a += 5;      // a = a + 5 → 15
3 a -= 3;      // a = a - 3 → 12
4 a *= 2;      // a = a * 2 → 24
5 a /= 4;      // a = a / 4 → 6
6 a %= 5;      // a = a % 5 → 1
```

运算符优先级

优先级	运算符	结合方向
最高	() 括号	—
↓	! ++ -- 正负号	右→左
↓	* / %	左→右
↓	+ -	左→右
↓	< <= > >=	左→右
↓	== !=	左→右
↓	&&	左→右
↓		左→右
最低	= += -= 等	右→左

 **建议：**不确定优先级时，**加括号！** 代码更清晰，也避免出错。

单元练习 2.1

练习1：输入圆的半径，输出面积和周长（保留两位小数）。

输入示例：

输出示例：

练习2：输入一个三位数，输出它的百位、十位、个位。

输入示例：

输出示例：

练习3：输入两个整数 a 和 b，输出 a 除以 b 的商和余数。

输入示例：

输出示例：

第三单元：流程控制结构

3.1 条件分支

if 语句

C/C++

```
1 int score;
2 cin >> score;
3
4 if (score >= 60) {
5     cout << "及格" << endl;
6 }
```

if-else 语句

C/C++

```
1 int score;
2 cin >> score;
3
4 if (score >= 60) {
5     cout << "及格" << endl;
6 } else {
7     cout << "不及格" << endl;
8 }
```

if-else if-else 语句

C/C++

```
1 int score;
2 cin >> score;
3
4 if (score >= 90) {
5     cout << "A" << endl;
6 } else if (score >= 80) {
7     cout << "B" << endl;
8 } else if (score >= 70) {
9     cout << "C" << endl;
10 } else if (score >= 60) {
11     cout << "D" << endl;
12 } else {
13     cout << "F" << endl;
14 }
```

嵌套条件

C/C++

```
1 int x, y;
2 cin >> x >> y;
3
4 if (x > 0) {
5     if (y > 0) {
6         cout << "第一象限" << endl;
7     } else if (y < 0) {
8         cout << "第四象限" << endl;
9     } else {
10        cout << "x轴正半轴" << endl;
11    }
12 } else if (x < 0) {
13     if (y > 0) {
14         cout << "第二象限" << endl;
15     } else if (y < 0) {
16         cout << "第三象限" << endl;
17     } else {
18         cout << "x轴负半轴" << endl;
19     }
20 } else {
21     if (y > 0) {
22         cout << "y轴正半轴" << endl;
23     } else if (y < 0) {
24         cout << "y轴负半轴" << endl;
25     } else {
26         cout << "原点" << endl;
27     }
28 }
```

三元运算符

C/C++

```
1 int a = 10, b = 20;
2 int max_val = (a > b) ? a : b;    // 如果 a>b 取 a, 否则取 b
3 cout << max_val << endl;        // 输出 20
```

switch-case 语句

C/C++

```
1 int day;
2 cin >> day;
3
4 switch (day) {
5     case 1: cout << "周一" << endl; break;
6     case 2: cout << "周二" << endl; break;
7     case 3: cout << "周三" << endl; break;
8     case 4: cout << "周四" << endl; break;
9     case 5: cout << "周五" << endl; break;
10    case 6: cout << "周六" << endl; break;
11    case 7: cout << "周日" << endl; break;
12    default: cout << "无效输入" << endl;
13 }
```

 **注意：** case 后面的 break 不能忘！忘记 break 会导致"贯穿"到下一个 case。

3.2 循环结构

while 循环

C/C++

```
1 // 输出 1 到 10
2 int i = 1;
3 while (i <= 10) {
4     cout << i << " ";
5     i++;
6 }
7 cout << endl;
```

for 循环

C/C++

```
1 // 输出 1 到 10
2 for (int i = 1; i <= 10; i++) {
3     cout << i << " ";
4 }
5 cout << endl;
6
7 // 输出 10 到 1 (递减)
8 for (int i = 10; i >= 1; i--) {
9     cout << i << " ";
10 }
11 cout << endl;
12
13 // 步长为2: 输出偶数
14 for (int i = 2; i <= 10; i += 2) {
15     cout << i << " "; // 2 4 6 8 10
16 }
```

break 和 continue

C/C++

```
1 // break: 跳出整个循环
2 for (int i = 1; i <= 10; i++) {
3     if (i == 5) break; // 当 i=5 时跳出循环
4     cout << i << " "; // 输出: 1 2 3 4
5 }
6
7 // continue: 跳过本次循环, 进入下一次
8 for (int i = 1; i <= 10; i++) {
9     if (i % 2 == 0) continue; // 跳过偶数
10    cout << i << " "; // 输出: 1 3 5 7 9
11 }
```

循环嵌套

C/C++

```
1 // 输出九九乘法表
2 for (int i = 1; i <= 9; i++) {
3     for (int j = 1; j <= i; j++) {
4         cout << j << "x" << i << "=" << i * j << "\t";
5     }
6     cout << endl;
7 }
```

do-while 循环

C/C++

```
1 // 至少执行一次
2 int i = 1;
3 do {
4     cout << i << " ";
5     i++;
6 } while (i <= 10);
7 cout << endl;
```

读取不确定数量的输入

C/C++

```
1 // 方法一: while + cin
2 int x;
3 while (cin >> x) {
4     cout << x * 2 << endl;
5 }
6 // 在终端按 Ctrl+Z (Windows) 或 Ctrl+D (Linux) 结束输入
7
8 // 方法二: 已知数量时
9 int n;
10 cin >> n;
11 for (int i = 0; i < n; i++) {
12     int x;
13     cin >> x;
14     // 处理 x
15 }
```

单元练习 3.1

练习1: 判断闰年。输入年份，输出是否为闰年（能被4整除但不能被100整除，或能被400整除）。

输入示例: 2024

输出示例: 2024是闰年

练习2: 输出 1~100 中所有能被 3 整除但不能被 5 整除的数。

输出示例: 3 6 9 12 18 21 ... 99

练习3: 打印如下三角形（输入行数 n）:

```
*
**
***
****
```

输入: 4

第四单元：数据结构与容器类型

4.1 数组（Array）

一维数组

C/C++

```
1 // 定义方式1: 指定大小
2 int arr1[10];           // 10个元素, 下标 0~9
3
4 // 定义方式2: 初始化列表
5 int arr2[5] = {1, 2, 3, 4, 5};
6
7 // 定义方式3: 不指定大小 (由初始化列表决定)
8 int arr3[] = {1, 2, 3, 4, 5}; // 编译器自动推断大小为5
9
10 // 定义方式4: 部分初始化 (其余为0)
11 int arr4[10] = {1, 2, 3};     // [1, 2, 3, 0, 0, 0, 0, 0, 0, 0]
```

数组访问与修改

C/C++

```
1 int arr[5] = {10, 20, 30, 40, 50};
2
3 // 访问 (下标从0开始)
4 cout << arr[0] << endl; // 10
5 cout << arr[4] << endl; // 50
6
7 // 修改
8 arr[2] = 100;           // [10, 20, 100, 40, 50]
9 cout << arr[2] << endl; // 100
```

 **数组越界：** `arr[5]` 在 5 元素数组中是越界的！C++ 不会自动检查，越界访问可能导致程序崩溃或数据损坏。

数组遍历

C/C++

```
1 int arr[5] = {10, 20, 30, 40, 50};
2
3 // 方式1: 传统 for 循环
4 for (int i = 0; i < 5; i++) {
5     cout << arr[i] << " ";
6 }
7 cout << endl;
8
9 // 方式2: 基于范围的 for 循环 (C++11+)
10 for (int x : arr) {
11     cout << x << " ";
12 }
13 cout << endl;
```

数组求和、最值

C/C++

```
1 int arr[100];
2 int n;
3 cin >> n;
4
5 int sum = 0;
6 int mx = INT_MIN; // 需要 #include <climits>
7 int mn = INT_MAX;
8
9 for (int i = 0; i < n; i++) {
10     cin >> arr[i];
11     sum += arr[i];
12     if (arr[i] > mx) mx = arr[i];
13     if (arr[i] < mn) mn = arr[i];
14 }
15
16 cout << "总和=" << sum << endl;
17 cout << "最大值=" << mx << endl;
18 cout << "最小值=" << mn << endl;
```

字符数组与字符串

C/C++

```
1 // C风格字符串（字符数组）
2 char str1[100] = "Hello";
3 char str2[100];
4 cin >> str2;           // 输入字符串（遇空格停止）
5
6 // C++ string 类（推荐）
7 string s1 = "Hello";
8 string s2;
9 cin >> s2;             // 输入（遇空格停止）
10 getline(cin, s2);     // 读取整行（含空格）
11
12 // 字符串操作
13 string s = "Hello World";
14 cout << s.length() << endl;    // 长度：11
15 cout << s[0] << endl;         // 第一个字符：H
16 cout << s.substr(0, 5) << endl; // 子串：Hello
```

⚠ **混用 cin 和 getline:** `cin >> n` 后缓冲区残留换行符，`getline` 会读到空字符串。解决方法：`cin.ignore()` 吃掉换行符。

4.2 STL 容器

vector（动态数组）

C/C++

```
1 #include <vector>
2 using namespace std;
3
4 // 创建
5 vector<int> v1;           // 空 vector
6 vector<int> v2(10);       // 10个元素, 初始为0
7 vector<int> v3(10, 5);    // 10个元素, 初始为5
8 vector<int> v4 = {1, 2, 3, 4, 5}; // 初始化列表
9
10 // 常用操作
11 v1.push_back(10);         // 尾部添加: O(1)
12 v1.push_back(20);
13 v1.push_back(30);
14 v1.pop_back();           // 尾部删除: O(1)
15
16 cout << v1.size() << endl; // 大小: 2
17 cout << v1[0] << endl;    // 访问: 10
18 cout << v1.at(1) << endl; // 安全访问 (越界抛异常)
19
20 // 遍历
21 for (int x : v1) {
22     cout << x << " ";
23 }
24 cout << endl;
25
26 // 排序
27 sort(v1.begin(), v1.end()); // 从小到大
28 sort(v1.begin(), v1.end(), greater<int>()); // 从大到小
```

pair (键值对)

C/C++

```
1 #include <utility>
2
3 pair<int, string> p1 = {1, "Alice"};
4 pair<int, int> p2(10, 20);
5
6 cout << p1.first << endl;          // 1
7 cout << p1.second << endl;        // Alice
8
9 // 常用于 map 和函数返回多值
```

map / unordered_map (键值映射)

C/C++

```
1 #include <map>
2 #include <unordered_map>
3
4 // map: 有序 (红黑树), 查找  $O(\log n)$ 
5 map<string, int> scores;
6 scores["Alice"] = 95;
7 scores["Bob"] = 87;
8 scores["Charlie"] = 92;
9
10 cout << scores["Alice"] << endl; // 95
11
12 // 遍历 (按 key 排序)
13 for (auto& kv : scores) {
14     cout << kv.first << ": " << kv.second << endl;
15 }
16
17 // unordered_map: 无序 (哈希表), 查找  $O(1)$ 
18 unordered_map<string, int> fast_scores;
19 fast_scores["Alice"] = 95;
```

set / unordered_set (集合)

C/C++

```
1 #include <set>
2 #include <unordered_set>
3
4 // set: 有序, 自动去重
5 set<int> s;
6 s.insert(30);
7 s.insert(10);
8 s.insert(20);
9 s.insert(10); // 重复, 不会插入
10
11 for (int x : s) {
12     cout << x << " "; // 输出: 10 20 30 (自动排序)
13 }
14
15 // 查找
16 if (s.count(10)) cout << "找到10" << endl;
```

常用 STL 算法

C/C++

```
1 #include <algorithm>
2
3 vector<int> v = {5, 2, 8, 1, 9};
4
5 // 排序
6 sort(v.begin(), v.end());           // 从小到大
7 reverse(v.begin(), v.end());        // 反转
8
9 // 查找
10 int pos = find(v.begin(), v.end(), 8) - v.begin(); // 查找8的位置
11 if (pos < v.size()) cout << "找到, 位置=" << pos << endl;
12
13 // 最值
14 int mx = *max_element(v.begin(), v.end()); // 最大值
15 int mn = *min_element(v.begin(), v.end()); // 最小值
16
17 // 计数
18 int cnt = count(v.begin(), v.end(), 5);    // 统计5出现次数
19
20 // 快速交换
21 swap(a, b);
22
23 // 求和
24 int sum = accumulate(v.begin(), v.end(), 0);
```

4.3 字符串进阶

C/C++

```
1 string s = "Hello World";
2
3 // 常用方法
4 cout << s.length() << endl;      // 长度: 11
5 cout << s.size() << endl;       // 同上
6 cout << s.empty() << endl;      // 是否为空: 0
7
8 // 拼接
9 string s2 = s + "!";             // "Hello World!"
10 s += " C++";                    // "Hello World C++"
11
12 // 比较
13 if (s == "Hello") cout << "相等" << endl;
14 if (s < "World") cout << "字典序小" << endl;
15
16 // 查找
17 size_t pos = s.find("World");    // 返回位置, 找不到返回 string::npos
18 if (pos != string::npos) cout << "找到, 位置=" << pos << endl;
19
20 // 替换
21 s.replace(0, 5, "Hi");           // 从位置0开始, 替换5个字符 → "Hi World"
22
23 // 截取子串
24 string sub = s.substr(0, 5);     // "Hello"
25
26 // 字符串转数字
27 string num_str = "12345";
28 int num = stoi(num_str);         // 12345
29 long long big = stoll(num_str);  // long long 版本
30
31 // 数字转字符串
32 int val = 42;
33 string str = to_string(val);     // "42"
34
35 // 格式化输出 (C风格)
36 char buffer[100];
37 sprintf(buffer, "pi=%.2f", 3.14159); // "pi=3.14"
```

单元练习 4.1

练习1：输入 n 个整数，用 vector 存储，输出去重后的元素个数和去重后的序列（保持原顺序）。

输入示例：

```
8
1 2 2 3 3 3 4 5
```

输出示例：

```
5
1 2 3 4 5
```

练习2：输入一个字符串，统计每个字符出现的次数，按字符ASCII顺序输出。

输入示例： aabcc

输出示例：

```
a:2
b:1
c:2
```

第五单元：函数与模块化

5.1 函数定义与调用

C/C++

```
1 // 基本语法
2 返回类型 函数名(参数列表) {
3     // 函数体
4     return 返回值; // 如果返回类型不是 void
5 }
6
7 // 示例：求两数之和
8 int add(int a, int b) {
9     return a + b;
10 }
11
12 // 调用
13 int result = add(3, 5); // result = 8
```

5.2 参数类型

位置参数（默认）

C/C++

```
1 int power(int base, int exp) {
2     int result = 1;
3     for (int i = 0; i < exp; i++) {
4         result *= base;
5     }
6     return result;
7 }
8
9 power(2, 10); // 1024
```

默认参数

C/C++

```
1 void printInfo(string name, int age = 18) {
2     cout << "姓名: " << name << ", 年龄: " << age << endl;
3 }
4
5 printInfo("Alice");           // 姓名: Alice, 年龄: 18
6 printInfo("Bob", 20);         // 姓名: Bob, 年龄: 20
```

⚠ 默认参数必须从右往左设置，且只能放在函数声明或定义中的一处。

引用参数（修改实参）

C/C++

```
1 // 值传递（不影响原变量）
2 void swap_val(int a, int b) {
3     int temp = a;
4     a = b;
5     b = temp;
6 }
7
8 // 引用传递（修改原变量）
9 void swap_ref(int& a, int& b) {
10    int temp = a;
11    a = b;
12    b = temp;
13 }
14
15 int x = 10, y = 20;
16 swap_ref(x, y); // x=20, y=10
```

可变参数

C/C++

```
1 #include <cstdarg>
2
3 int sum(int count, ...) {
4     va_list args;
5     va_start(args, count);
6
7     int total = 0;
8     for (int i = 0; i < count; i++) {
9         total += va_arg(args, int);
10    }
11
12    va_end(args);
13    return total;
14 }
15
16 sum(3, 10, 20, 30); // 60
```

5.3 返回值

C/C++

```
1 // 返回单个值
2 int max(int a, int b) {
3     return (a > b) ? a : b;
4 }
5
6 // 返回多个值: 使用 pair 或 struct
7 pair<int, int> findMinMax(vector<int>& v) {
8     int mn = *min_element(v.begin(), v.end());
9     int mx = *max_element(v.begin(), v.end());
10    return {mn, mx};
11 }
12
13 // 使用
14 auto [minimum, maximum] = findMinMax(v); // C++17 结构化绑定
```

5.4 作用域

C/C++

```
1 int global_var = 100;    // 全局变量（不推荐在竞赛中滥用）
2
3 void func() {
4     int local_var = 50;  // 局部变量
5     cout << global_var << endl;  // 可以访问全局
6     cout << local_var << endl;
7 }
8
9 int main() {
10     // cout << local_var << endl;  // 错误！无法访问 func 的局部变量
11     cout << global_var << endl;    // 可以访问全局
12     return 0;
13 }
```

⚠ 竞赛建议：

全局数组大小设为常量：`const int MAXN = 100005; int arr[MAXN];`

避免不必要的全局变量，减少命名冲突

如需在函数间共享大数组，可定义为全局

5.5 函数递归（基础）

C/C++

```
1 // 递归求阶乘
2 int factorial(int n) {
3     if (n <= 1) return 1;        // 终止条件
4     return n * factorial(n - 1); // 递归调用
5 }
6
7 cout << factorial(5) << endl;    // 120
```

5.6 Lambda 表达式（C++11+）

C/C++

```
1 // 基本语法: [捕获列表](参数) -> 返回类型 { 函数体 }
2
3 // 简单 lambda
4 auto greet = []() { cout << "Hello!" << endl; };
5 greet();
6
7 // 带参数和返回值
8 auto add = [](int a, int b) -> int { return a + b; };
9 cout << add(3, 5) << endl; // 8
10
11 // 捕获外部变量
12 int base = 10;
13 auto add_base = [base](int x) { return x + base; };
14 cout << add_base(5) << endl; // 15
15
16 // 在 sort 中使用 lambda (自定义排序)
17 vector<int> v = {5, 2, 8, 1, 9};
18 sort(v.begin(), v.end(), [](int a, int b) {
19     return a > b; // 从大到小
20 });
```

5.7 头文件与模块化

C/C++

```
1 // ===== mylib.h (头文件) =====
2 #ifndef MYLIB_H
3 #define MYLIB_H
4
5 int add(int a, int b);
6 int subtract(int a, int b);
7 const double PI = 3.1415926535;
8
9 #endif
10
11 // ===== mylib.cpp (实现文件) =====
12 #include "mylib.h"
13
14 int add(int a, int b) {
15     return a + b;
16 }
17
18 int subtract(int a, int b) {
19     return a - b;
20 }
21
22 // ===== main.cpp (主程序) =====
23 #include <iostream>
24 #include "mylib.h"
25 using namespace std;
26
27 int main() {
28     cout << add(10, 3) << endl;    // 13
29     cout << subtract(10, 3) << endl; // 7
30     return 0;
31 }
```

💡 **竞赛中：**一般把所有代码写在一个 `.cpp` 文件中，用 `#include <bits/stdc++.h>` 引入所有库即可。

单元练习 5.1

练习1: 写一个函数 `bool isPrime(int n)` 判断 n 是否为素数。

输入示例: 17

输出示例: true

练习2: 写一个函数 `int gcd(int a, int b)` 用辗转相除法求最大公约数。

输入示例: 48 18

输出示例: 6

第六单元：文件与异常处理

6.1 文件读写

文本文件写入

C/C++

```
1 #include <fstream>
2
3 // 写文件
4 ofstream outfile("output.txt");
5 if (outfile.is_open()) {
6     outfile << "Hello File!" << endl;
7     outfile << "第二行内容" << endl;
8     outfile.close();
9 }
```

文本文件读取

C/C++

```
1 // 读文件
2 ifstream infile("input.txt");
3 if (infile.is_open()) {
4     string line;
5     while (getline(infile, line)) {
6         cout << line << endl;
7     }
8     infile.close();
9 }
```

竞赛中的文件操作

C/C++

```
1 int main() {
2     // 从文件读取输入（竞赛常用技巧）
3     freopen("input.txt", "r", stdin);
4     freopen("output.txt", "w", stdout);
5
6     int a, b;
7     cin >> a >> b;
8     cout << a + b << endl;
9
10    fclose(stdin);
11    fclose(stdout);
12    return 0;
13 }
```

6.2 异常处理

C/C++

```
1 #include <stdexcept>
2
3 // 基本语法
4 try {
5     // 可能抛出异常的代码
6     int denominator = 0;
7     if (denominator == 0) {
8         throw runtime_error("除数不能为零!");
9     }
10    cout << 10 / denominator << endl;
11 } catch (const runtime_error& e) {
12    cout << "捕获异常: " << e.what() << endl;
13 } catch (...) {
14    cout << "捕获未知异常" << endl;
15 }
16
17 // 常见标准异常
18 try {
19    vector<int> v(5);
20    cout << v.at(10) << endl;    // 越界, 抛出 out_of_range
21 } catch (const out_of_range& e) {
22    cout << "越界: " << e.what() << endl;
23 }
```

单元练习 6.1

练习: 编写程序, 从 `data.txt` 文件中读取若干整数, 计算它们的和与平均值, 将结果写入 `result.txt`。

`data.txt` 内容示例:

10 20 30 40 50

`result.txt` 期望内容:

总和=150

平均值=30.00

基础模块 • 课后作业

作业1：编写一个程序，输入一个年份，判断是否为闰年并输出结果。如果是闰年输出 "Leap Year"，否则输出 "Not Leap Year"。

作业2：输入一个正整数 n，输出 1+2+3+...+n 的和。要求用循环实现。

作业3：输入一个字符串，判断它是否是回文（正读反读一样）。忽略大小写。

作业4：用数组存储 10 个学生的成绩（0~100），输出最高分、最低分、平均分，以及高于平均分的人数。

作业5：编写函数 `void printDiamond(int n)` 打印一个 n 行的菱形图案。

输入：5

输出：

```
  *
 ***
*****
 ***
  *
```

第二部分：竞赛进阶

第五单元：排序算法

5.1 冒泡排序（Bubble Sort）

算法原理

核心思想：相邻两个元素两两比较，如果顺序不对就交换。每一轮将当前最大（或最小）的元素"冒泡"到正确位置。

执行过程演示（排序 `[5, 3, 8, 1, 2]` 从小到大）：

Plain Text

```
1 初始: [5, 3, 8, 1, 2]
2
3 第1轮:
4   5>3 → 交换 → [3, 5, 8, 1, 2]
5   5<8 → 不交换 → [3, 5, 8, 1, 2]
6   8>1 → 交换 → [3, 5, 1, 8, 2]
7   8>2 → 交换 → [3, 5, 1, 2, 8] ← 8 已就位
8
9 第2轮:
10  3<5 → 不交换
11  5>1 → 交换 → [3, 1, 5, 2, 8]
12  5>2 → 交换 → [3, 1, 2, 5, 8] ← 5 已就位
13
14 第3轮:
15  3>1 → 交换 → [1, 3, 2, 5, 8]
16  3>2 → 交换 → [1, 2, 3, 5, 8] ← 3 已就位
17
18 第4轮:
19  1<2 → 不交换 → [1, 2, 3, 5, 8] ← 完成!
```

代码实现

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void bubbleSort(int arr[], int n) {
5     for (int i = 0; i < n - 1; i++) {
6         bool swapped = false; // 优化标志
7         for (int j = 0; j < n - 1 - i; j++) {
8             if (arr[j] > arr[j + 1]) {
9                 swap(arr[j], arr[j + 1]);
10                swapped = true;
11            }
12        }
13        if (!swapped) break; // 已经有序，提前结束
14    }
15 }
16
17 int main() {
18     int n;
19     cin >> n;
20     int arr[105];
21     for (int i = 0; i < n; i++) {
22         cin >> arr[i];
23     }
24
25     bubbleSort(arr, n);
26
27     for (int i = 0; i < n; i++) {
28         cout << arr[i] << " ";
29     }
30     cout << endl;
31     return 0;
32 }
```

复杂度分析

指标	值
时间复杂度（最坏 / 平均）	$O(n^2)$
时间复杂度（最好）	$O(n)$ （已排序时，加优化）
空间复杂度	$O(1)$ （原地排序）
稳定性	 稳定（相等元素不交换）

Example 5.1

题目：输入 n 个整数，用冒泡排序从小到大排序后输出。

输入：

Plain Text

1 5
2 5 3 8 1 2

输出：

Plain Text

1 1 2 3 5 8

完整解答：

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(0);
7
8     int n;
9     cin >> n;
10    int arr[105];
11
12    for (int i = 0; i < n; i++) {
13        cin >> arr[i];
14    }
15
16    // 冒泡排序
17    for (int i = 0; i < n - 1; i++) {
18        for (int j = 0; j < n - 1 - i; j++) {
19            if (arr[j] > arr[j + 1]) {
20                swap(arr[j], arr[j + 1]);
21            }
22        }
23    }
24
25    for (int i = 0; i < n; i++) {
26        cout << arr[i] << " ";
27    }
28    cout << endl;
29
30    return 0;
31 }
```

5.2 桶排序 (Bucket Sort / Counting Sort)

算法原理

核心思想：创建一个"桶数组"，遍历原始数据，将每个元素放入对应编号的桶中（计数），最后按桶的顺序依次取出。

适用场景：数据范围已知且不太大（如 0~100 的成绩排序）。

执行过程演示（排序 `[50, 30, 80, 10, 20]`）：

Plain Text

```
1 创建101个桶（0~100），初始全为0
2 ↓
3 遍历数据：
4   50 → 桶[50]++
5   30 → 桶[30]++
6   80 → 桶[80]++
7   10 → 桶[10]++
8   20 → 桶[20]++
9 ↓
10 桶状态：桶[10]=1，桶[20]=1，桶[30]=1，桶[50]=1，桶[80]=1
11 ↓
12 从桶[0]到桶[100]依次输出：
13   10, 20, 30, 50, 80
```

代码实现

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void bucketSort(int arr[], int n, int max_val = 100) {
5     int buckets[105] = {0}; // 初始化全为0
6
7     // 计数阶段
8     for (int i = 0; i < n; i++) {
9         buckets[arr[i]]++;
10    }
11
12    // 输出阶段
13    int idx = 0;
14    for (int i = 0; i <= max_val; i++) {
15        while (buckets[i] > 0) {
16            arr[idx++] = i;
17            buckets[i]--;
18        }
19    }
20 }
21
22 int main() {
23     int n;
24     cin >> n;
25     int arr[105];
26     for (int i = 0; i < n; i++) {
27         cin >> arr[i];
28     }
29
30     bucketSort(arr, n);
31
32     for (int i = 0; i < n; i++) {
33         cout << arr[i] << " ";
34     }
35     cout << endl;
36     return 0;
37 }
```


复杂度分析

指标	值
时间复杂度	$O(n + k)$ ，k 为桶的数量
空间复杂度	$O(k)$
稳定性	 稳定

Example 5.2

题目：输入 n 个 0~100 之间的整数，用桶排序从小到大排序后输出。

输入：

Plain Text

1 5
2 50 30 80 10 20

输出：

Plain Text

1 10 20 30 50 80

完整解答：

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(0);
7
8     int n;
9     cin >> n;
10    int buckets[101] = {0}; // 桶数组, 初始全0
11
12    for (int i = 0; i < n; i++) {
13        int x;
14        cin >> x;
15        buckets[x]++; // 对应桶计数+1
16    }
17
18    // 按桶顺序输出
19    for (int i = 0; i <= 100; i++) {
20        while (buckets[i] > 0) {
21            cout << i << " ";
22            buckets[i]--;
23        }
24    }
25    cout << endl;
26
27    return 0;
28 }
```

单元练习 5.1 (课堂练习)

练习1: 用冒泡排序对字符串数组按字典序从小到大排序。

输入: apple banana pear peach grape

输出: apple banana grape peach pear

练习2: 有 1000000 个 0~9 之间的随机数, 用桶排序统计每个数字出现的次数, 按数字从小到大输出。

输入: 10 (然后输入10个0~9的数字)

示例输入：10\n3 1 4 1 5 9 2 6 5 3

输出：

0:0

1:2

2:1

3:2

4:1

5:2

6:1

7:0

8:0

9:1

课后作业 5

作业1：实现冒泡排序的降序版本（从大到小排序）。

作业2：输入 n 个学生的姓名和成绩（成绩 0~100），用桶排序按成绩从高到低输出学生姓名（成绩相同按输入顺序）。

输入示例：

5

Alice 85

Bob 92

Charlie 78

David 92

Eve 85

输出示例：

Bob David Alice Eve Charlie

第六单元：深度优先搜索（DFS）

6.1 DFS 基本原理

核心思想：一条路走到黑，走不通就回退一步，换条路继续走。类似走迷宫时"一路向右手边走"的策略。

实现方式：递归（最自然）或显式栈

与 BFS 的对比：

特性	DFS	BFS
数据结构	栈（递归 / 显式栈）	队列
空间复杂度	$O(\text{深度})$	$O(\text{宽度})$
适合场景	找所有解、路径搜索	找最短路径
实现方式	递归（最自然）	队列循环

6.2 全排列问题

Example 6.1：生成全排列

题目：输入 n ，输出 $1\sim n$ 的所有排列。

输入：

输出：

Plain Text

```
1 1 2 3
2 1 3 2
3 2 1 3
4 2 3 1
5 3 1 2
6 3 2 1
```

完整解答：

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n;
5 vector<int> path;          // 当前路径
6 vector<bool> used;        // 标记是否使用过
7
8 void dfs() {
9     // 终止条件：路径长度等于n
10    if (path.size() == n) {
11        for (int i = 0; i < n; i++) {
12            cout << path[i] << " ";
13        }
14        cout << endl;
15        return;
16    }
17
18    // 尝试每个数字
19    for (int i = 1; i <= n; i++) {
20        if (!used[i]) {          // 如果数字 i 还没被使用
21            used[i] = true;      // 标记为已使用
22            path.push_back(i);   // 加入路径
23            dfs();               // 递归搜索
24            path.pop_back();     // 回溯：撤销选择
25            used[i] = false;     // 回溯：标记为未使用
26        }
27    }
28 }
29
30 int main() {
31     cin >> n;
32     used.resize(n + 1, false);
33     dfs();
34     return 0;
35 }
```

执行过程追踪 (n=3):

Plain Text

```
1 dfs()
2 |─ i=1: used[1]=T, path=[1]
3 |   |─ i=2: used[2]=T, path=[1,2]
4 |   |   |─ i=3: used[3]=T, path=[1,2,3] → 输出!
5 |   |   └─ 回溯: path=[1,2], used[3]=F
6 |   |─ i=3: used[3]=T, path=[1,3]
7 |   |   |─ i=2: used[2]=T, path=[1,3,2] → 输出!
8 |   └─ 回溯
9 |─ i=2: ...
10 └─ i=3: ...
```

关键三步骤（DFS 模板）：

- 1. **选择**：标记已访问，加入路径
- 2. **递归**：深入下一层
- 3. **回溯**：撤销选择，恢复状态

6.3 迷宫问题

Example 6.2：迷宫路径判定

题目：在 $n \times m$ 的迷宫中，从起点 $(0,0)$ 到终点 $(n-1, m-1)$ ，只能上下左右走，求是否存在路径（0=通路，1=墙壁）。

完整解答：

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m;
5 vector<vector<int>> maze;
6 vector<vector<bool>> visited;
7
8 bool dfs(int x, int y) {
9     // 到达终点
10    if (x == n - 1 && y == m - 1) {
11        return true;
12    }
13
14    visited[x][y] = true; // 标记为已访问
15
16    // 四个方向: 上、右、下、左
17    int dx[] = {-1, 0, 1, 0};
18    int dy[] = {0, 1, 0, -1};
19
20    for (int dir = 0; dir < 4; dir++) {
21        int nx = x + dx[dir];
22        int ny = y + dy[dir];
23
24        // 检查边界、可通行、未访问
25        if (nx >= 0 && nx < n && ny >= 0 && ny < m
26            && maze[nx][ny] == 0 && !visited[nx][ny]) {
27            if (dfs(nx, ny)) {
28                return true;
29            }
30        }
31    }
32
33    visited[x][y] = false; // 回溯
34    return false;
35 }
36
37 int main() {
38     cin >> n >> m;
39     maze.resize(n, vector<int>(m));
```

```

40     visited.resize(n, vector<bool>(m, false));
41
42     for (int i = 0; i < n; i++) {
43         for (int j = 0; j < m; j++) {
44             cin >> maze[i][j];
45         }
46     }
47
48     if (dfs(0, 0)) {
49         cout << "有路径! " << endl;
50     } else {
51         cout << "无路径! " << endl;
52     }
53
54     return 0;
55 }

```

单元练习 6.1 (课堂练习)

练习1: 输入 n , 输出 $1 \sim n$ 中所有和为 n 的组合 (每个数只能用一次, 顺序不同算同一种)。

输入: 5

输出:

1 4

2 3

5

练习2: 在 $n \times n$ 的棋盘上放置 n 个皇后, 使它们互不攻击 (不在同一行、列、对角线)。输出一种可行方案。

课后作业 6

作业1: 用 DFS 解决"岛屿数量"问题。给定一个 0/1 矩阵, 1 表示陆地, 0 表示水, 计算岛屿数量 (上下左右相连的陆地算一个岛)。

输入:

4 4

1 1 0 0

1 0 0 1

0 0 1 1

0 0 1 0

输出: 3

作业2：用 DFS 找出图中从起点到终点的所有路径。

第七单元：广度优先搜索（BFS）

7.1 BFS 基本原理

核心思想：从起点开始，先访问所有距离为 1 的点，再访问距离为 2 的点……逐层扩展。类似水波扩散。

实现方式：队列（Queue）

7.2 BFS 模板

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // BFS 通用模板
5 void bfs(int start) {
6     queue<int> q;
7     vector<bool> visited(N, false);
8
9     q.push(start);
10    visited[start] = true;
11
12    while (!q.empty()) {
13        int current = q.front();
14        q.pop();
15
16        // 处理当前节点
17        cout << current << " ";
18
19        // 遍历邻居
20        for (int neighbor : get_neighbors(current)) {
21            if (!visited[neighbor]) {
22                visited[neighbor] = true;
23                q.push(neighbor);
24            }
25        }
26    }
27 }
```

7.3 Example 7.1: 最短路径（迷宫）

题目：在 $n \times m$ 的迷宫中，0 表示通路，1 表示墙壁，求从 (0,0) 到 (n-1,m-1) 的最短步数。

输入：

Plain Text

```
1 5 5
2 0 1 0 0 0
3 0 1 0 1 0
4 0 0 0 1 0
5 1 1 0 1 0
6 0 0 0 0 0
```

输出：

8

完整解答：

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m;
5 vector<vector<int>> maze;
6 vector<vector<bool>> visited;
7
8 int bfs() {
9     // queue 存储 (x, y, steps)
10    queue<pair<int, int>> q;
11    visited[0][0] = true;
12    q.push({0, 0});
13
14    int steps = 0;
15    int dx[] = {-1, 0, 1, 0};
16    int dy[] = {0, 1, 0, -1};
17
18    while (!q.empty()) {
19        int size = q.size();
20
21        for (int i = 0; i < size; i++) {
22            auto [x, y] = q.front();
23            q.pop();
24
25            // 到达终点
26            if (x == n - 1 && y == m - 1) {
27                return steps;
28            }
29
30            // 四个方向
31            for (int dir = 0; dir < 4; dir++) {
32                int nx = x + dx[dir];
33                int ny = y + dy[dir];
34
35                if (nx >= 0 && nx < n && ny >= 0 && ny < m
36                    && maze[nx][ny] == 0 && !visited[nx][ny]) {
37                    visited[nx][ny] = true;
38                    q.push({nx, ny});
39                }
40            }
41        }
42        steps++;
43    }
44}
```

```

40         }
41     }
42     steps++;
43 }
44
45     return -1; // 无法到达
46 }
47
48 int main() {
49     cin >> n >> m;
50     maze.resize(n, vector<int>(m));
51     visited.resize(n, vector<bool>(m, false));
52
53     for (int i = 0; i < n; i++) {
54         for (int j = 0; j < m; j++) {
55             cin >> maze[i][j];
56         }
57     }
58
59     cout << bfs() << endl;
60     return 0;
61 }

```

7.4 Example 7.2: 洪水填充 (Flood Fill)

题目：给定一个矩阵，从指定起点开始，将与起点连通的所有相同值区域标记为新值。

完整解答：

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void floodFill(vector<vector<int>>& grid, int sr, int sc, int newColor) {
5     int rows = grid.size();
6     int cols = grid[0].size();
7     int originalColor = grid[sr][sc];
8
9     if (originalColor == newColor) return;
10
11     queue<pair<int, int>> q;
12     q.push({sr, sc});
13     grid[sr][sc] = newColor;
14
15     int dx[] = {-1, 0, 1, 0};
16     int dy[] = {0, 1, 0, -1};
17
18     while (!q.empty()) {
19         auto [r, c] = q.front();
20         q.pop();
21
22         for (int dir = 0; dir < 4; dir++) {
23             int nr = r + dx[dir];
24             int nc = c + dy[dir];
25
26             if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
27                 && grid[nr][nc] == originalColor) {
28                 grid[nr][nc] = newColor;
29                 q.push({nr, nc});
30             }
31         }
32     }
33 }
```

单元练习 7.1 (课堂练习)

练习1: 在一个 8×8 的棋盘上，马（骑士）从 (0,0) 出发，求到达 (7,7) 的最少步数。（马走"日"字：横2竖1或横1竖2）

练习2: 给定一个字符串和一本"字典"（字符串列表），每次可以变换字符串中的一个字符，且变换后的字符串必须在字典中。求从起始字符串到目标字符串的最短变换序列长度。

课后作业 7

作业1: 在 $n \times m$ 的网格中，有些格子有障碍物（1表示障碍，0表示空地）。从 (0,0) 出发，每次可以上下左右走，求到达 (n-1,m-1) 的最少步数。如果无法到达，输出 -1。

作业2: 用 BFS 解决"水壶问题"：有两个容量分别为 a 和 b 的水壶，求能否量出恰好 c 升水。

第八单元：递归

8.1 递归基本原理

核心思想：函数调用自身来解决问题。大问题分解为同类小问题。

递归三要素：

- 终止条件**（Base Case）：什么时候不再递归
- 递归调用**：如何把问题分解为更小的子问题
- 返回结果**：如何将子问题的结果组合成最终答案

8.2 经典递归问题

Example 8.1：斐波那契数列

题目：求斐波那契数列的第 n 项（ $F(1)=1, F(2)=1, F(n)=F(n-1)+F(n-2)$ ）。

方法一：朴素递归（理解原理，效率低）

C/C++

```
1 int fib(int n) {
2     if (n <= 2) return 1;           // 终止条件
3     return fib(n - 1) + fib(n - 2); // 递归调用
4 }
5
6 cout << fib(10) << endl;           // 55
```

⚠️ 朴素递归有大量重复计算，时间复杂度 $O(2^n)$ ， n 稍大就会超时！

方法二：记忆化递归（优化）

C/C++

```
1 #include <unordered_map>
2
3 unordered_map<int, long long> memo;
4
5 long long fib_memo(int n) {
6     if (n <= 2) return 1;
7     if (memo.count(n)) return memo[n]; // 已计算过，直接返回
8     memo[n] = fib_memo(n - 1) + fib_memo(n - 2);
9     return memo[n];
10 }
11
12 cout << fib_memo(50) << endl;         // 快速计算大数
```

方法三：递推 / 迭代（最高效）

C/C++

```
1 long long fib_iter(int n) {
2     if (n <= 2) return 1;
3     long long a = 1, b = 1;
4     for (int i = 3; i <= n; i++) {
5         long long temp = a + b;
6         a = b;
7         b = temp;
8     }
9     return b;
10 }
11
12 cout << fib_iter(100) << endl;           // 354224848179261915075
```

Example 8.2: 汉诺塔

题目：将 n 个盘子从柱子 A 移动到柱子 C，每次只能移动一个盘子，大盘不能放在小盘上。输出移动步骤。

完整解答：

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void hanoi(int n, char source, char target, char auxiliary) {
5     if (n == 1) {
6         cout << "将盘子1从" << source << "移到" << target << endl;
7         return;
8     }
9
10    // 将上面 n-1 个盘子从 source 移到 auxiliary
11    hanoi(n - 1, source, auxiliary, target);
12
13    // 将最大的盘子从 source 移到 target
14    cout << "将盘子" << n << "从" << source << "移到" << target << endl;
15
16    // 将 n-1 个盘子从 auxiliary 移到 target
17    hanoi(n - 1, auxiliary, target, source);
18 }
19
20 int main() {
21     hanoi(3, 'A', 'C', 'B');
22     return 0;
23 }
```

输出：

Plain Text

```
1 将盘子1从A移到C
2 将盘子2从A移到B
3 将盘子1从C移到B
4 将盘子3从A移到C
5 将盘子1从B移到A
6 将盘子2从B移到C
7 将盘子1从A移到C
```

递归分解图 (n=3)：

Plain Text

```
1 hanoi(3, A, C, B)
2  |— hanoi(2, A, B, C)
3  |   |— hanoi(1, A, C, B) → A→C
4  |   |— A→B
5  |   |— hanoi(1, C, B, A) → C→B
6  |— A→C
7  |— hanoi(2, B, C, A)
8     |— hanoi(1, B, A, C) → B→A
9     |— B→C
10    |— hanoi(1, A, C, B) → A→C
```

Example 8.3: 二分搜索（递归版）

C/C++

```
1 int binarySearch(int arr[], int target, int left, int right) {
2     if (left > right) return -1;           // 终止条件：找不到
3
4     int mid = left + (right - left) / 2;    // 防止溢出
5
6     if (arr[mid] == target) {
7         return mid;                        // 找到了
8     } else if (arr[mid] > target) {
9         return binarySearch(arr, target, left, mid - 1); // 左半部分
10    } else {
11        return binarySearch(arr, target, mid + 1, right); // 右半部分
12    }
13 }
14
15 // 测试
16 int main() {
17     int nums[] = {1, 3, 5, 7, 9, 11, 13};
18     int n = sizeof(nums) / sizeof(nums[0]);
19
20     cout << binarySearch(nums, 7, 0, n - 1) << endl;    // 3
21     cout << binarySearch(nums, 8, 0, n - 1) << endl;    // -1
22     return 0;
23 }
```

 **防止溢出技巧：** `mid = left + (right - left) / 2` 比 `(left + right) / 2` 更安全，避免大数相加溢出。

单元练习 8.1（课堂练习）

练习1：用递归实现求 n 的阶乘。

输入：5

输出：120

练习2：用递归实现字符串反转。

输入：hello

输出：olleh

练习3：一只青蛙一次可以跳 1 级或 2 级台阶，问跳上 n 级台阶有多少种方法？用递归实现。

课后作业 8

- 作业1：**用递归实现求最大公约数（辗转相除法）。
提示： $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ ，当 $b=0$ 时 $\text{gcd}=a$ 。
- 作业2：**用递归判断一个字符串是否是回文。
- 作业3：**用递归解决"子集生成"问题：给定一个集合，输出它所有的子集。

第九单元：递推

9.1 递推基本原理

核心思想：从已知初始条件出发，通过递推公式逐步计算出后续项。与递归方向相反——递归是从大问题到小问题，递推是从小问题到大问题。

递推 vs 递归：

特性	递推	递归
方向	从小到大	从大到小
实现	循环	函数调用自身
空间	$O(1)$ 或 $O(n)$	$O(n)$ 调用栈
效率	通常更高	可能有重复计算

9.2 经典递推问题

Example 9.1：爬楼梯

题目：一次可以走 1 级或 2 级台阶，上 n 级台阶有多少种走法？

分析：

- 到达第 n 级，要么从第 n-1 级走 1 步，要么从第 n-2 级走 2 步
- 递推公式： $\text{dp}[n] = \text{dp}[n-1] + \text{dp}[n-2]$

- 初始条件: `dp[1] = 1, dp[2] = 2`

完整解答:

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long climbStairs(int n) {
5     if (n <= 2) return n;
6
7     long long a = 1, b = 2; // dp[1]=1, dp[2]=2
8     for (int i = 3; i <= n; i++) {
9         long long temp = a + b;
10        a = b;
11        b = temp;
12    }
13    return b;
14 }
15
16 int main() {
17     for (int i = 1; i <= 6; i++) {
18         cout << i << "级台阶: " << climbStairs(i) << "种" << endl;
19     }
20     // 输出:
21     // 1级台阶: 1种
22     // 2级台阶: 2种
23     // 3级台阶: 3种
24     // 4级台阶: 5种
25     // 5级台阶: 8种
26     // 6级台阶: 13种
27     return 0;
28 }
```

Example 9.2: 最大公约数 (辗转相除法)

题目: 输入两个正整数 a 和 b , 求它们的最大公约数。

算法原理: $\gcd(a, b) = \gcd(b, a \% b)$, 直到 $b = 0$ 时 $\gcd = a$ 。

完整解答:

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // 方法一：循环（递推）
5 int gcd_iter(int a, int b) {
6     while (b != 0) {
7         int temp = b;
8         b = a % b;
9         a = temp;
10    }
11    return a;
12 }
13
14 // 方法二：递归
15 int gcd_recur(int a, int b) {
16     if (b == 0) return a;
17     return gcd_recur(b, a % b);
18 }
19
20 // 最小公倍数
21 int lcm(int a, int b) {
22     return a / gcd_iter(a, b) * b; // 先除后乘，防溢出
23 }
24
25 int main() {
26     cout << gcd_iter(48, 18) << endl; // 6
27     cout << gcd_recur(48, 18) << endl; // 6
28     cout << lcm(12, 18) << endl; // 36
29     return 0;
30 }
```

执行过程追踪 (gcd(48, 18)):

Plain Text

```
1 gcd(48, 18)
2 → 48 % 18 = 12 → gcd(18, 12)
3 → 18 % 12 = 6 → gcd(12, 6)
4 → 12 % 6 = 0 → gcd(6, 0)
5 → 返回 6
```

Example 9.3: 百钱买百鸡

题目：公鸡 5 元 / 只，母鸡 3 元 / 只，小鸡 1 元 / 3 只。用 100 元买 100 只鸡，有多少种方案？

分析：

- 设公鸡 x 只，母鸡 y 只，小鸡 z 只
- 条件 1: $x + y + z = 100$
- 条件 2: $5x + 3y + z/3 = 100$
- 条件 3: z 必须是 3 的倍数

完整解答：

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     cout << "百钱买百鸡的所有方案：" << endl;
6     int count = 0;
7
8     for (int x = 0; x <= 20; x++) {           // 公鸡最多20只
9         for (int y = 0; y <= 33; y++) {       // 母鸡最多33只
10             int z = 100 - x - y;             // 小鸡数量
11
12             if (z % 3 == 0 && 5 * x + 3 * y + z / 3 == 100) {
13                 count++;
14                 cout << "方案" << count << "：公鸡" << x
15                     << "只，母鸡" << y << "只，小鸡" << z << "只" << endl;
16             }
17         }
18     }
19
20     return 0;
21 }
```

输出：

Plain Text

- 1 方案1：公鸡0只，母鸡25只，小鸡75只
- 2 方案2：公鸡4只，母鸡18只，小鸡78只
- 3 方案3：公鸡8只，母鸡11只，小鸡81只
- 4 方案4：公鸡12只，母鸡4只，小鸡84只

Example 9.4：数字反转

题目：输入一个整数，输出它的反转数。

完整解答：

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     int original = n;
8     int rev = 0;
9
10    while (n > 0) {
11        int last_digit = n % 10;    // 取最后一位
12        rev = rev * 10 + last_digit; // 构建反转数
13        n /= 10;                    // 去掉最后一位
14    }
15
16    cout << original << "的反转数是: " << rev << endl;
17    // 12345 → 54321
18
19    return 0;
20 }
```

单元练习 9.1 (课堂练习)

练习1: 用递推方法求 $1! + 2! + 3! + \dots + n!$ 的和。

输入:

输出: ($1!+2!+3!+4!+5! = 1+2+6+24+120$)

练习2: 约瑟夫环问题: n 个人围成一圈, 从 1 开始报数, 数到 3 的人出局。求最后剩下的人最初的编号。

输入:

输出:

课后作业 9

作业1: 用递推实现"杨辉三角"前 n 行。

输入:

输出:



- 作业2：用递推解决"铺砖问题"：用 1×1 和 1×2 的砖铺满 $1\times n$ 的地面，有多少种铺法？
- 作业3：用辗转相除法和更相减损术分别实现最大公约数，对比两种方法的效率。

竞赛进阶模块 · 综合练习

- 综合题1：在一个 $n\times m$ 的网格中，从左上角到右下角，每次只能向右或向下走。求总共有多少种不同的路径？
- 综合题2：用 DFS + 剪枝解决"N皇后"问题：在 $n\times n$ 的棋盘上放置 n 个皇后，使它们互不攻击。输出所有可行方案的数量。
- 综合题3：给定一个无向图（邻接矩阵表示），用 BFS 判断图是否连通（从任意点出发能到达所有点）。

附录：竞赛常用技巧速查

A.1 输入输出优化

```
C/C++

1 // 关闭同步，加速 cin/cout（处理大数据必用）
2 ios::sync_with_stdio(false);
3 cin.tie(0);
4
5 // 使用 '\n' 而非 endl（endl 会刷新缓冲区，较慢）
6 cout << "Hello\n";    // 快
7 cout << "Hello" << endl; // 慢
```

A.2 常用宏定义

C/C++

```
1 #define MAXN 100005
2 #define INF 0x3f3f3f3f
3 #define ll long long
4 #define pb push_back
5 #define mp make_pair
6 #define fi first
7 #define se second
```

A.3 快速交换与最值

C/C++

```
1 swap(a, b); // 需要 #include <algorithm>
2 int mx = max(a, b); // 最大值
3 int mn = min(a, b); // 最小值
4 int abs_val = abs(-5); // 绝对值
```

A.4 数组初始化技巧

C/C++

```
1 int arr[105] = {0}; // 所有元素初始化为0
2 memset(arr, 0, sizeof(arr)); // 用 memset 清零
3 fill(arr, arr + 105, 0); // 用 fill 填充
```

A.5 常见错误速查

错误	说明
数组越界	<code>arr[5]</code> 在 5 元素数组中越界，C++ 不检查
忘记初始化	局部变量值是随机的
scanf 忘记 <code>&</code>	<code>scanf("%d", a)</code> ❌
double 用 <code>%f</code> 输入	必须用 <code>%lf</code>
整数除法	<code>5/2=2</code> ，需要 <code>5.0/2</code> 或 <code>(double)5/2</code>
混用 cin 和 getline	中间加 <code>cin.ignore()</code>

— 教材正文结束 —

新哲文院算法社 2026-2027 学年